



Order Flow

An event-driven e-commerce order-fulfillment reference architecture utilizing: Dot Net 10, Aspire, EF Core, and Angular. Built locally, priced at zero spend, but wired for real Azure cloud deployment.

Prepared by **Robert Felkins**

Principal Architect – Architect4Hire.com

Contents

Purpose & Positioning	3
Design Principles.....	3
Scope	4
In Scope.....	4
Out of Scope	4
System Overview.....	4
Logical Flow	4
Services and Responsibilities.....	8
Saga & Compensation.....	8
Forward Path	8
Compensation Path	8
Failure Handling & Resilience	8
Mechanisms.....	9
Data & Persistence.....	9
Messaging & Contracts	10
Representative messages	10
Front End.....	10
Observability.....	10
Architecture Decision Records	10
Build Phases.....	11
Open Questions.....	11

Purpose & Positioning

Overflow is a reference architecture, not a product. It exists to prove one thing to a prospective client: that the person they are considering hiring can design an event-driven system that behaves correctly when things go wrong – not just when the demo goes right.

The domain is deliberately universal. An order comes in, inventory is reserved, payment is charged, the warehouse is told to ship, and the customer is notified. Every business vertical: retail, distribution, manufacturing, or logistics, understands this flow without translation. That makes it the ideal canvas for showing architectural judgment rather than explaining it.

What is this document for

- A shared reference for the Order Flow system and build – service boundaries, contracts, failure behavior, and the decisions behind them.
- The planning artifact that the SCRUB prompt library is derived from, and the raw materials for the accompanying case study and blog posts.
- An editable draft. Everything here is a starting position and is meant to be fodder for conversation, not a settled specification.

Design Principles

These are the rules that design and build are held to. When a decision is contested later, it is settled by appeal to one of these principles.

Right-sizing is an architectural decision

The default is the boring, correct solution. Microservices, event sourcing, and a service mesh each solve a real problem, and each becomes a liability without that problem. Order Flow decomposes only where the seam earns its keep, and this document records what was deliberately left together.

Correct under failure, not just under demo

Every distributed interaction assumes the other side can be slow, absent, or duplicated. Idempotency, retries with backoff, compensation, and dead lettering are first-class elements, not afterthoughts.

Local by default, backed by a real SDK

The whole system runs on Aspire-orchestrated emulators with zero Azure spend, but it talks to real Azure SDKs. Going live is a configuration and DevOps change, not a rewrite. The demo is honest: no simulation shortcuts standing in for the real messaging or storage semantics.

Observability is part of the deliverable

If a behavior cannot be seen in a trace, it is not done. The Aspire dashboard is treated as a demo surface, not just a development environment convenience.

Scope

In Scope

- Order placement with one or more line items.
- Inventory reservation with correct concurrency behavior under contention.
- Simulated payment with idempotent processing and duplicate-callback protection.
- Fulfillment dispatch to a simulated warehouse/carrier, with transient failure retry.
- Best-effort customer notifications on state transitions.
- A saga coordinating the above, with compensation for failure.
- An Angular operations/status view and a minimal customer order view.
- Live distributed tracing via the Aspire dashboard and OpenTelemetry.
- A small set of Architecture Decision Records (ADRs).

Out of Scope

- Real payment or carrier integrations — simulated providers only.
- A production storefront, catalog browsing, cart, or checkout UX polish.
- Auth beyond a minimal stub is not the auth-modernization demo.
- A dedicated warehouse-management subsystem (explicitly not decomposed — see ADR-004).
- Multi-region, multi-tenant, or horizontal-scale concerns beyond what a trace can illustrate.

System Overview

Order Flow is a small set of services coordinated by a saga over an asynchronous message bus. The order service owns the workflow; Inventory, Payment, Fulfillment, and Notifications react to messages and report back. State transitions are recorded to an append-only order event log; a read model serves fast status queries.

Logical Flow

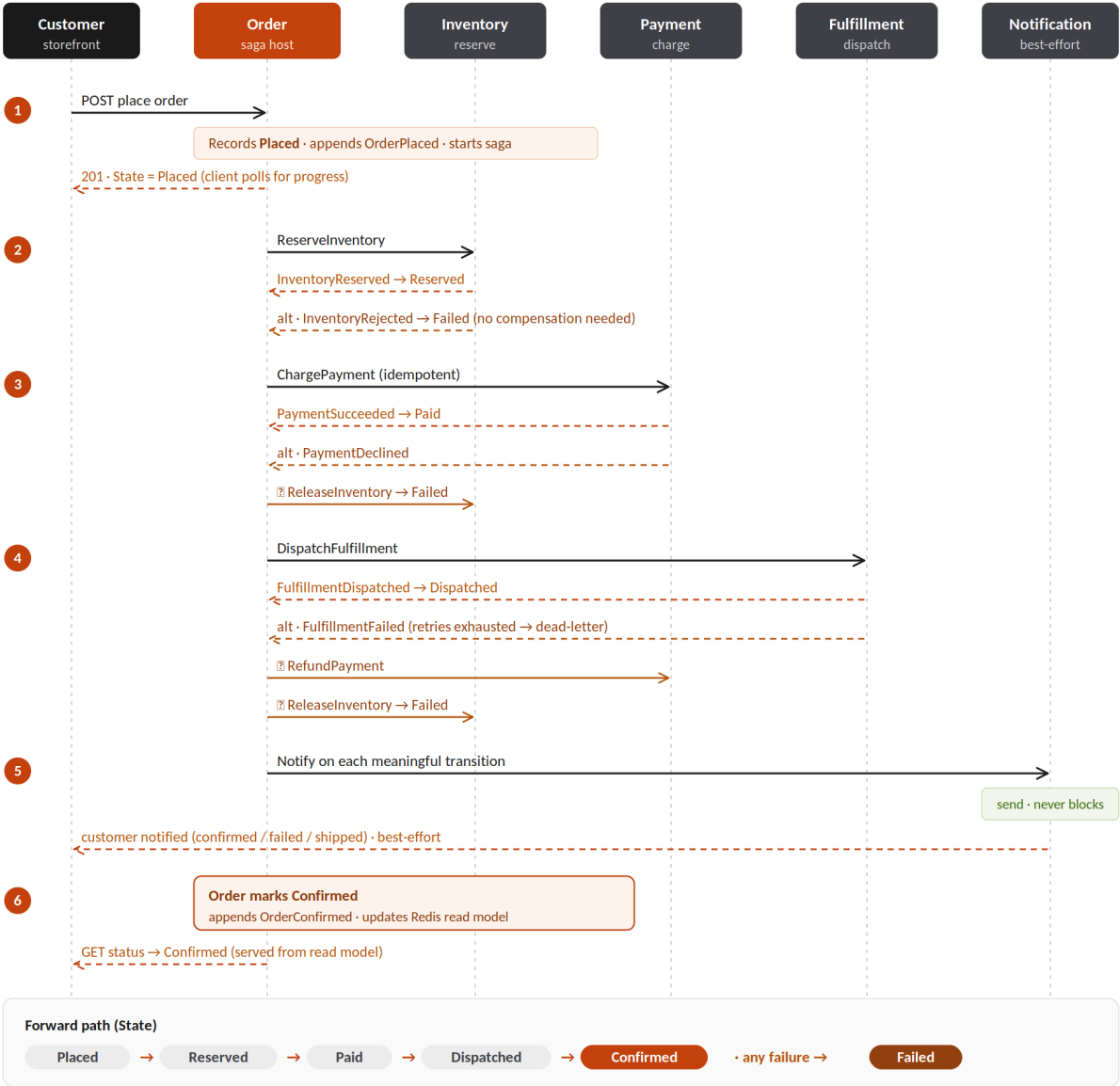
1. Customer places an order → Order services records it as Placed and starts the saga
2. Inventory reserves stock for each line → replies Reserved or Rejected
3. Payment charges the order total → replies Paid or Declined
4. Fulfillment dispatches to the warehouse → replies Dispatched
5. Notification informs the customer at each meaningful transition
6. Order service marks the order as Confirmed and updates the read model

ORDER FLOW HIGH-LEVEL DESIGN

OrderFlow — Logical Flow

Order placement through confirmation, with reply and failure branches.

→ command / call
- - -> failure branch
- - -> reply event
⚠ compensation

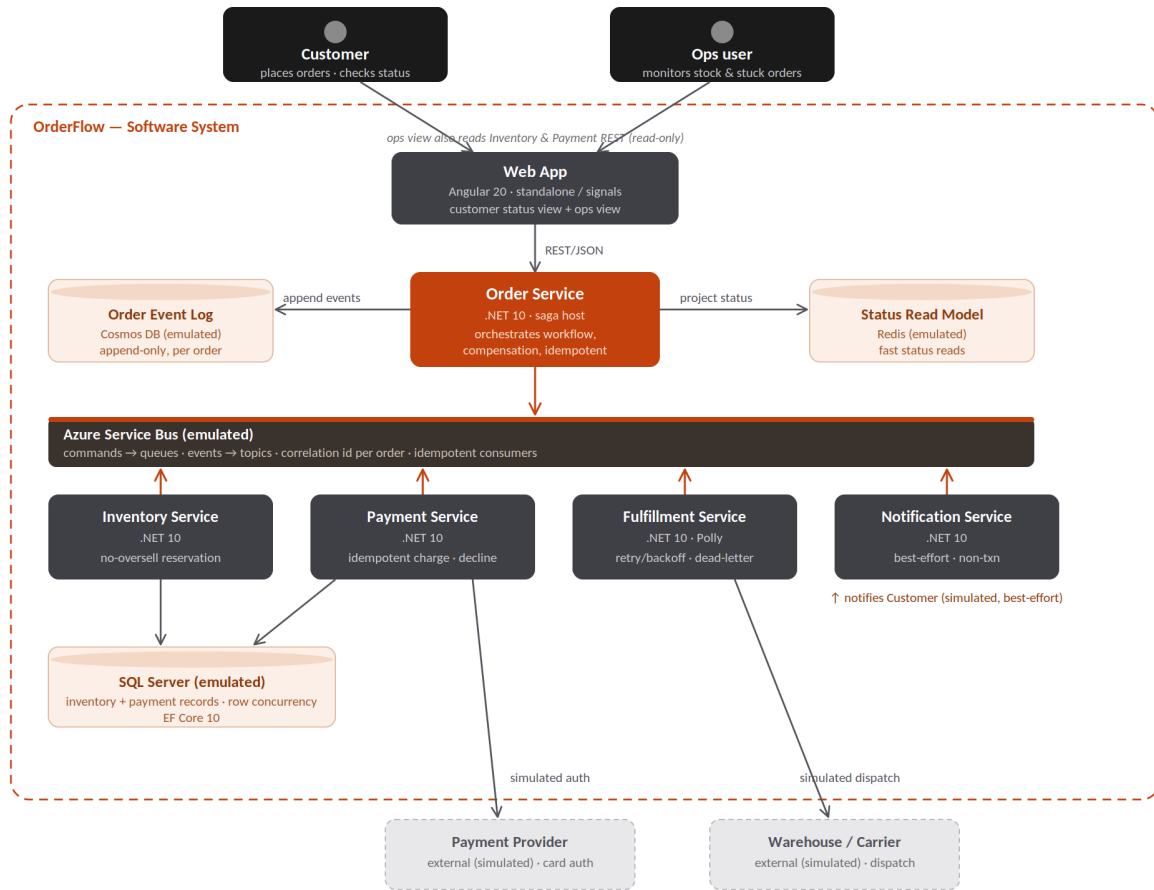


ORDER FLOW HIGH-LEVEL DESIGN

OrderFlow — C4 Container Diagram

Event-driven order fulfillment · .NET 10 / Aspire 13.3 / Angular 20 · Aspire-local emulators, real Azure SDKs.

● person ■ container ■ saga host
■ data store ■ external (sim) ■ async bus

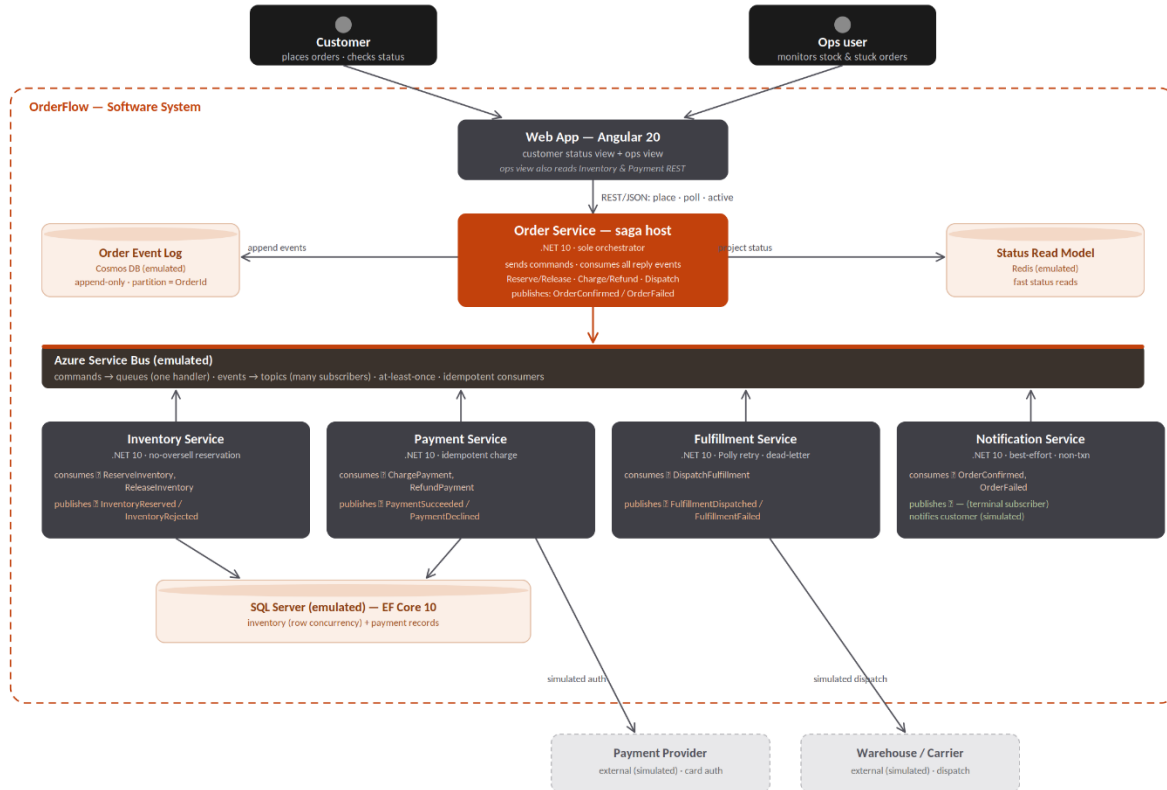


ORDER FLOW HIGH-LEVEL DESIGN

OrderFlow — C4 Container Diagram (Interactions)

What each container consumes and publishes. All inter-service messaging is asynchronous over Azure Service Bus, one correlation id per order.

→ command (consumed) → REST / SDK
- - - - - event (published)



Services and Responsibilities

Service	Owns	Key behavior it proves
Order	Order aggregate, workflow state, event log	Orchestration, compensation, idempotent transitions
Inventory	On hand / reserved / available per SKU	Concurrency-correct reservation no oversell
Payment	Charging attempts and outcomes	Idempotency against duplicate callbacks; decline handling
Fulfillment	Dispatch requests to the warehouse/carrier	Retry with backoff; dead letter on hard failure
Notification	Customer messages on transitions	Best effort; never blocks or rolls back the order

Saga & Compensation

The order workflow is a saga: a sequence of local transactions where each step has a compensating action that undoes it. There are no distributed transactions and no 2-phase commits – those are rejected deliberately in ADR-001.

Forward Path

Placed → Reserved → Paid → Dispatched → Confirmed

Compensation Path

- Payment declines after inventory is reserved → release the reservation, mark the order Failed with a customer-facing reason
- Fulfillment fails permanently after payment → refund the charge, release the reservation, and mark as Failed
- Inventory rejects up front → no compensation needed; mark as Failed immediately

Failure Handling & Resilience

This is the section the whole build serves. Each scenario below is meant to be demonstrated live, ideally by killing a dependency mid-flow and narrating the recovery.

Scenario	Expected Behavior
Duplicate payment callback	Second callback is a no-op; no double charge, no duplicate transition.
Concurrent purchase of the last unit	Exactly one reservation wins; the other fails cleanly. No oversell.

Payment gateway down	Retry with backoff via a circuit breaker; recover when it returns.
Carrier calls fail permanently	Message dead-lettered; order enters a visible stuck state with a reason.
Payment fails after reservation	Reservation released (compensation); order marked Failed.
Notification provider down	Order proceeds unaffected; notification retried best-effort, then dropped.
Poison message on the bus	Dead-lettered after N attempts; visible to ops, replayable.

Mechanisms

- **Idempotency** – A durable idempotency key per external callback and per message consumer, so at least once delivery is safe
- **Retries & Circuit Breaking** – Polly policies on outbound gateway/carrier calls, tuned per dependency rather than one global default.
- **Dead Lettering** – Poison and hard-fail messages land in a DLQ with enough context to diagnose and replay.
- **Concurrency Control** – Optimistic concurrency on inventory rows; the losing writer fails and is handled, never silently overwritten.

Data & Persistence

Store	Data Type	Why
Cosmos DB	Append-only order event log	Natural fit for an immutable event stream; see ADR-002
SQL DB	Inventory, payment records.	Strong consistency and row-level concurrency where it matters
Redis	Order status read model	Fast status reads without hitting the event log every time

This is a lightweight CQRS split: writes go through the domain and the event log; reads are served from a projection. It is intentionally not full event sourcing – the read model is a convenience, not the system of record for every aggregate. That boundary is deliberate and documented.

Messaging & Contracts

Services communicate over Azure Service Bus (emulated locally). Commands are directed to a single handler; events are published for any interested subscriber. Every message carries a correlation id so a single order can be followed end-to-end in a trace.

Representative messages

- Commands: ReserveInventory, ChargePayment, DispatchFulfillment, ReleaseInventory, RefundPayment.
- Events: OrderPlaced, InventoryReserved, InventoryRejected, PaymentSucceeded, PaymentDeclined, FulfillmentDispatched, FulfillmentFailed, OrderConfirmed, OrderFailed.

Front End

Angular 20, reusing the conventions established in AspireCafe2026 so the portfolio reads as one coherent stack rather than three unrelated demos. Two surfaces, both intentionally minimal:

- **Customer order view** — place an order and watch its status advance in real time through the saga states.
- **Ops view** — inventory levels per SKU, payment attempt history, and a list of stuck / dead-lettered orders with their reasons.

Observability

OpenTelemetry traces, metrics, and logs flow into the Aspire dashboard. The demo-critical capability is following one order across every service as a single distributed trace — placement through reservation, payment, dispatch, and confirmation — including the retry and compensation paths when a failure is injected.

- Every message and handler is on the same correlation id / trace.
- Dead-lettered messages are visible with their failure reason and attempt count.
- Key business metrics (orders confirmed, failed, compensated) are emitted as counters.

Architecture Decision Records

Each ADR is a short record of a decision, the alternatives considered, and why they were rejected. These are where "right-sizing is an architectural decision" stops being a slogan and becomes visible. The full ADRs live alongside the code; summaries follow.

ADR	Decision	Rejected Alternative
ADR-001	Saga with compensation for the order workflow	Two-phase commit / distributed transaction
ADR-002	Cosmos DB for the append-only order event log	Single relational store for everything
ADR-003	Light CQRS (read model) for order status	Full event sourcing of every aggregate

ADR-004	No separate warehouse-management service	Decomposing fulfillment into its own subsystem
ADR-005	Aspire-local emulation with real Azure SDKs	Provisioning live Azure for the demo

Build Phases

Rough sequencing. Each phase should end with something demonstrable, in keeping with the “ship in slices” approach.

1. **Skeleton** - Aspire AppHost, service projects, contracts library, dashboard wired up. Happy path order placement end to end, no resilience yet
2. **Correctness** – Idempotency, concurrency-correct inventory, saga compensation on the payment-decline path
3. **Resilience** – Polly policies, dead-lettering, the full failure matrix from Failure Handling & Resilience
4. **Observability & UI** – Traces polished for demo, ops view, stuck order visibility
5. **Documentation** – ADRs finalized, Readme.md and the case study post drafted from this document.

Open Questions

1. Confirm the canonical name – Order Flow is a placeholder
2. Is Notification worth a full service, or does it collapse into Order?
3. How much of the auth stub is worth building vs deferring entirely to the auth-modernization demo?
4. Do we want a scripted “chaos” toggle in the UI to trigger failures live, or drive failures from a test harness?